MATH10017: R Programming Exercises

Contents

Short Questions **1**
- Question 1: Vectors and indexing . . . 1
- Question 2: Vectorised computation . . . 1
- Question 3: Matrices and row/column operations . . . 2
- Question 4: Writing a simple function . . . 2
- Question 5: Using `apply` instead of a loop . . . 2
- Question 6: Data frames: creating, filtering, selecting . . . 2
- Question 7: Graphics in base R . . . 3
- Question 8: Lists and `sapply` . . . 3
- Question 9: Logical indexing with `my_which` . . . 3

Longer Questions **3**
- Question 1: Matrix operations and linear algebra . . . 3
- Question 2: Writing and using your own function with `sapply` . . . 4
- Question 3: Data frames and summary statistics . . . 4
- Question 4: Base graphics with multiple layers . . . 5
- Question 5: Using `apply` on a matrix of statistics . . . 5
- Question 6 (advanced): Functional programming with `lapply` and data frames . . . 5
- Question 7: Vector creation, indexing, and summary . . . 6
- Question 8: Working with sequences and proportions . . . 6
- Question 9: Vectorised unit conversions and labels . . . 6
- Question 10: Recycling . . . 6
- Question 11: Matrix creation and column summaries . . . 7
- Question 12: Submatrices and summary statistics . . . 7
- Question 13: Z-score function . . . 7
- Question 14: Weighted mean function . . . 7

Short Questions

Question 1: Vectors and indexing

Create a numeric vector `x` containing the integers from 1 to 20.
Using *one line of code*, extract all elements of `x` that are multiples of 3 and store them in `y`.

Solution

Question 2: Vectorised computation

You have a vector of temperatures in Celsius:

```
temp_C <- c(0, 10, 20, 30)
```

Write *vectorised* R code (no explicit `for` loop) to convert these to Fahrenheit and store them in `temp_F`. (Use the formula F = 9/5 * C + 32.)

Solution

Question 3: Matrices and row/column operations

Create a 3×4 matrix `M` containing the integers from 1 to 12 (filled by rows).

Then:

1. Compute the sum of each row (a length-3 vector).
2. Extract the second column of `M` as a vector.

Solution

Question 4: Writing a simple function

Write an R function `my_mean` that takes a numeric vector `x` and returns its mean **without** using the built-in `mean()` function.
(You may assume `x` has length > 0.)

Solution

Question 5: Using `apply` instead of a loop

Let

```r
A <- matrix(1:9, nrow = 3)  # 3×3 matrix
```

Use `apply()` to compute the **sum of each column** of `A` (i.e. a length-3 vector) and store it in `col_sums`. Do not use `colSums()`.

Solution

Question 6: Data frames: creating, filtering, selecting

Create a data frame `students` with three columns:

- `name` = `c("Alice", "Bob", "Cara", "Dan")`
- `age` = `c(19, 20, 18, 21)`
- `mark` = `c(72, 55, 88, 64)`

Then:

1. Select only the rows for students with `mark >= 60`.
2. From that filtered data frame, extract only the `name` and `mark` columns.

Solution

Question 7: Graphics in base R

Given:

```
x <- 0:10
y <- x^2
```

Write R code to:

1. Plot `y` against `x` using points.
2. Add a **line** connecting the points on the same plot.

Solution

Question 8: Lists and `sapply`

Consider the list:

```
L <- list(
  a = 1:3,
  b = 10:15,
  c = c(5, 5, 5, 5)
)
```

Use `sapply()` to compute the **length** of each element of `L`, returning a numeric vector of lengths.

Solution

Question 9: Logical indexing with `my_which`

Create a function `my_which` that takes as input a logical vector `v` and returns the indices of all `TRUE` elements (similar to R's built-in `which()` function).

Then test your function on:

`v <- c(TRUE, FALSE, TRUE, TRUE, FALSE)`

Then consider the numeric vector:

```
z <- c(3, 8, 1, 8, 5)
```

Use `my_which()` to find the **indices** of all elements equal to 8, and store them in `idx`.
Then use `idx` to replace those elements with 0 (modifying `z`).

Solution

Longer Questions

Question 1: Matrix operations and linear algebra

Let

```r
A <- matrix(c(1, 2, 3,
              0, 1, 4,
              5, 6, 0), nrow = 3, byrow = TRUE)
b <- c(7, 4, 10)
```

1. Compute the transpose of `A`.
2. Compute the matrix product `A %*% b`.
3. Solve the linear system `A x = b` for `x` by computing $x = A^{-1}b$ (use `solve()`).

Solution

Question 2: Writing and using your own function with `sapply`

Write a function `is_prime` that takes a single positive integer `n` and returns `TRUE` if `n` is prime, `FALSE` otherwise.

Then:

1. Use `is_prime` on the numbers from 1 to 20.
2. Use `sapply(1:20, is_prime)` to get a logical vector of primality.
3. Use this logical vector to extract all primes between 1 and 20.

(You may use a simple, not highly optimised, algorithm.)

Solution

Question 3: Data frames and summary statistics

You have exam data for a class:

```r
students <- data.frame(
  name   = c("Alice", "Bob", "Cara", "Dan", "Eli"),
  gender = c("F", "M", "F", "M", "M"),
  mark1  = c(72, 55, 88, 64, 90),
  mark2  = c(68, 60, 91, 70, 85)
)
```

1. Add a new column `avg_mark` which is the mean of `mark1` and `mark2` for each student.
2. Find the **overall** mean of `avg_mark`.
3. Use logical indexing to extract the rows of students with `avg_mark >= 70`.

Solution

Question 4: Base graphics with multiple layers

Suppose you have data from a simple physics experiment:

```
time <- seq(0, 10, by = 0.5)
distance <- 3 * time + rnorm(length(time), mean = 0, sd = 2)
```

1. Plot `distance` (y-axis) against `time` (x-axis) using points.
2. Add a straight line showing the **true** underlying relationship `distance = 3 * time` in a different plotting layer.
3. Add a main title and axis labels.

Solution

Question 5: Using `apply` on a matrix of statistics

Create a 4×5 matrix `M` containing the integers 1 to 20 (filled by rows).

Then:

1. Use `apply()` to find the **mean of each row**.
2. Use `apply()` to find the **maximum of each column**.
3. Combine these results into a list with named components `row_means` and `col_max`.

Solution

Question 6 (advanced): Functional programming with `lapply` and data frames

This is an **advanced** question.

Create a list `L` containing three small data frames, each with a single numeric column:

```
set.seed(1)
L <- list(
  df1 = data.frame(x = rnorm(5, mean = 0, sd = 1)),
  df2 = data.frame(x = rnorm(5, mean = 10, sd = 2)),
  df3 = data.frame(x = rnorm(5, mean = -5, sd = 1))
)
```

1. Use `lapply()` to compute the mean of column `x` for each data frame in the list.
2. Convert the result into a numeric vector using `unlist()`.
3. Add a new column `x_centered` to each data frame, where you subtract its own mean from each value of `x`. (Hint: use `lapply()` again.)

Solution

Question 7: Vector creation, indexing, and summary

You are given daily production outputs (in kg) from a small factory over 30 days, stored as a sequence from 12 to 41:

```
prod <- 12:41
```

1. Extract all production values greater than 30.
2. Count how many days exceeded 30 kg production.
3. Compute the mean production **only among the days where production was greater than 30**.

Solution

Question 8: Working with sequences and proportions

Create a vector `x` containing 100 values equally spaced from -π to +π (`pi` in R).

1. Print the first 6 and last 6 elements.
2. Use logical indexing to extract values of `x` between -1 and +1.
3. Compute the proportion of values in this interval.

Solution

Question 9: Vectorised unit conversions and labels

The vector below represents raw heights measured in **metres** for a sample of 6 people:

```
h <- c(1.55, 1.72, 1.68, 1.81, 1.60, 1.90)
```

1. Convert these heights to **centimetres**.
2. Convert them to **inches** (1 inch = 2.54 cm).
3. Create a vector indicating `"tall"` for heights > 1.75 m and `"not tall"` otherwise.

Solution

Question 10: Recycling

Consider:

```
x <- c(3, 6, 9, 12, 15, 18)
y <- c(1, 2)
```

1. Compute `x + y`.
2. Explain in a few sentences how R handled the shorter vector `y` during this operation.

Solution

Explanation (to write in words): R automatically recycles `y` *to the length of* `x`*, turning it into* `c(1, 2, 1, 2, 1, 2)` *and then adds element-wise.*

Question 11: Matrix creation and column summaries

Create a **5×4 matrix** `M` that contains the integers 1 to 20 (filled by **rows**).

1. Extract the **first and third rows** of `M`.
2. Compute the **sum of each column** using `apply()` (do not use `colSums()`).
3. Multiply the matrix by 2 (vectorised multiplication).

Solution

Question 12: Submatrices and summary statistics

Using the matrix `M` from:

1. Extract the **submatrix of rows 3-4 and columns 2-3**.
2. Square each element of this submatrix and call the resulting matrix V.
3. Compute the inverse of $C = VV$.

Solution

Question 13: Z-score function

Write a function `z_score()` that calculates the vector

$$z = \frac{x - \mu}{\sigma}$$

where `x` is a numeric vector, and `mu` and `sigma` are scalars.

1. Apply the function to `x <- c(10, 12, 9, 11, 15)` using `mu = 11` and `sigma = 2`.
2. Print the result on the R console.

Solution

Question 14: Weighted mean function

Create a function `weighted_mean(x, w)` that computes the weighted mean of a numeric vector `x` with weights `w` using the formula:

$$\text{weighted mean} = \frac{\sum_{i=1}^{d} x_i w_i}{\sum_{j=1}^{d} w_j}$$

where d is the length of `x` and `w`.

1. Test it on

```
x <- c(60, 75, 80)
w <- c(1, 2, 1)
```

2. Check the result of `weighted_mean(x, w)` against those of `weighted.mean(x, w)` (R default function for doing this).

Solution
